

Decision and Risk Analysis

Lecture 7: **Approximating the Posterior**

Grid Approximation, MCMC, and RStan

DAEN 427 / ISEN 427 / ISEN 627 - Fall 2026

Alexander Abuabara



1. **The problem:** the posterior is often impossible to write down
2. **Grid approximation:** brute force, easy to see
3. **MCMC:** a smart random walk that scales
4. **Diagnostics:** how to know the simulation can be trusted
5. **Decisions:** turn posterior draws into risk numbers

Example

A new component design is tested.

Unknown: its failure probability π .

Decision: **keep the design or replace it?**

Why do we need to approximate?

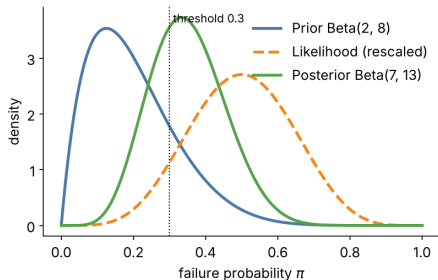
$$\begin{array}{|c|} \hline \text{Prior} \\ f(\pi) \\ \hline \end{array} \times \begin{array}{|c|} \hline \text{Likelihood} \\ L(\pi | y) \\ \hline \end{array} \propto \begin{array}{|c|} \hline \text{Posterior} \\ f(\pi | y) \\ \hline \end{array}$$

$$f(\pi | y) = \frac{f(\pi) L(\pi | y)}{\int f(\pi) L(\pi | y) d\pi}$$

- Numerator: easy, we just multiply two functions.
- Denominator: an integral, **hard** for non-conjugate or many-parameter models.
- Risk analysis needs numbers such as $P(\pi > \text{threshold} | y)$ and expected loss.
- We do not need the formula. We need **a large sample of plausible π values** drawn from the posterior.

Example: component failure probability

- Prior from experience:
 $\pi \sim \text{Beta}(2, 8)$
(typically about 20%)
- Test: $Y = 5$ failures
in $n = 10$ units
- Model: $Y \mid \pi \sim \text{Bin}(10, \pi)$
- Here the answer is known:
 $\text{Beta}(7, 13)$.



Why start with a solved problem? So we can *check* the approximations against the truth.

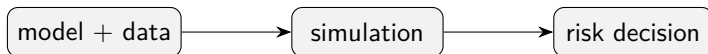
Two approximation ideas

1. Grid approximation

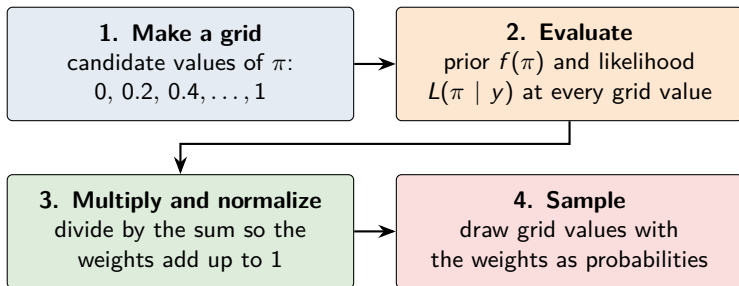
- Try many possible parameter values.
- Score each value by $\text{prior} \times \text{likelihood}$.
- Normalize scores into probabilities.
- Sample from the discrete grid.

2. MCMC

- Build a dependent sequence of values.
- Each next value depends on the current value.
- A good chain mimics the posterior.
- RStan handles the sampling algorithm.



Grid approximation



Think of it as a **spreadsheet**: one row per candidate π , columns for prior, likelihood, product, and normalized weight.

Grid approximation by hand (6 values)

```
pi_grid    <- seq(from = 0, to = 1, length = 6)      # (1)
prior      <- dbeta(pi_grid, shape1 = 2, shape2 = 8) # (2)
likelihood <- dbinom(x = 5, size = 10, prob = pi_grid)
unnormalized <- prior * likelihood                  # (3)
posterior  <- unnormalized / sum(unnormalized)
```

π	prior	likelihood	prior $\times$ lik.	posterior
0.0	0.000	0.000	0.000	0.000
0.2	3.020	0.026	0.080	0.312
0.4	0.806	0.201	0.162	0.632
0.6	0.071	0.201	0.014	0.056
0.8	0.001	0.026	0.000	0.000
1.0	0.000	0.000	0.000	0.000

- (1) grid of candidate π .
- (2) dbeta, dbinom evaluate the prior and likelihood at all grid points at once.
- (3) Dividing by the sum makes the weights a probability distribution.

Grid approximation in R

```
#  $Y \mid \pi \sim \text{Bin}(10, \pi)$ ,  $\pi \sim \text{Beta}(2, 2)$ , observed  $Y = 9$ 

library(tidyverse)

# Step 1: choose grid points
grid_data <- data.frame(pi_grid = seq(0, 1, length = 6))

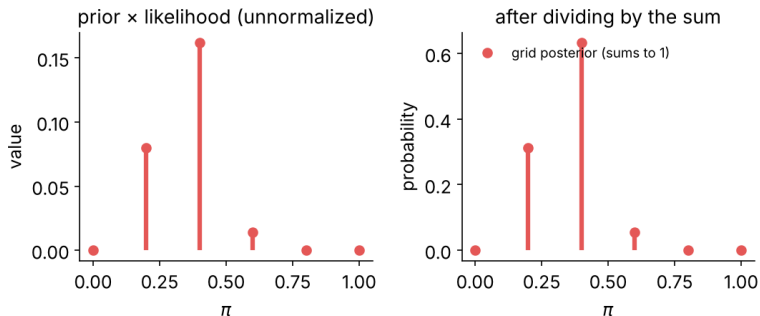
# Step 2: evaluate prior and likelihood on the grid
grid_data <- grid_data %>%
  mutate(prior = dbeta(pi_grid, 2, 2),
         likelihood = dbinom(9, size = 10, prob = pi_grid))

# Step 3: prior  $\times$  likelihood, then normalize
grid_data <- grid_data %>%
  mutate(unnormalized = prior * likelihood,
         posterior = unnormalized / sum(unnormalized))
```

Key coding idea: each row is one possible value of π .

The posterior column is a probability distribution across rows.

What the 6-point grid looks like

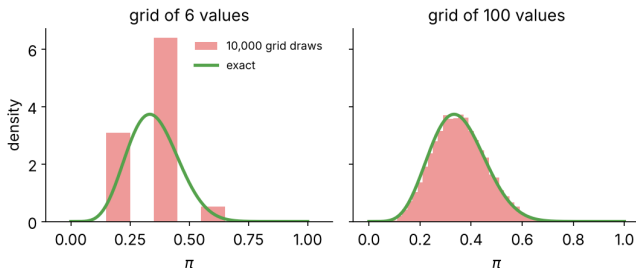


- Only $\pi = 0.2$ and 0.4 carry real weight: too coarse to see the shape.
- A coarse grid can **misplace risk**: it has no value between 0.2 and 0.4 .
- Remedy: a finer grid (next slide).

Sample from the grid and compare with the truth

```
grid101 <- grid_post(101)  # same code as before, 101 points
draws <- sample(grid101$pi_grid, size = 10000, replace = TRUE,
                prob = grid101$posterior)  # (4)

ggplot(data.frame(pi = draws), aes(pi)) +
  geom_histogram(aes(y = after_stat(density)), bins = 50) +
  stat_function(fun = dbeta, args = list(shape1 = 7, shape2 = 13))
```

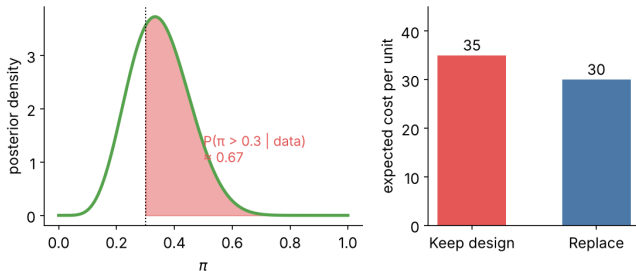


(4) `sample()` turns weights into a Monte Carlo sample. With 101 points the histogram follows the exact Beta(7,13) curve.

From draws to a risk decision

```
mean(draws)                # posterior mean of pi
mean(draws > 0.3)          # P(pi > 0.3 | data)
quantile(draws, c(0.025, 0.975)) # 95% credible interval

cost_fail <- 100; cost_repl <- 30          # cost per unit
c(keep = cost_fail * mean(draws), replace = cost_repl)
```



Reading it: $P(\pi > 0.3 | y) \approx 0.65\text{--}0.67$; expected cost of keeping = $100 \times 0.35 = 35 > 30$. Under this loss, **replace the design**.

Another model (Gamma–Poisson, failures per month)

Plant has λ failures/month. Prior $\lambda \sim \text{Gamma}(3, 1)$.

Three months of data: $y = (2, 4, 3)$.

```
y      <- c(2, 4, 3)
lam_grid <- seq(0, 12, length = 200)
prior   <- dgamma(lam_grid, shape = 3, rate = 1)
lik     <- sapply(lam_grid,
                  function(l) prod(dpois(y, l))) # (5)
post    <- prior * lik / sum(prior * lik)
lam_draws <- sample(lam_grid, 10000, replace = TRUE, prob = post)
```

(5) Independent months: the likelihood is a *product* of Poisson probabilities.

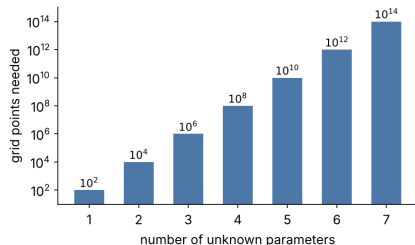
Same four steps, different model. The recipe does not change.

Grid vs exact (Gamma–Poisson)



- Exact posterior: $\text{Gamma}(3 + 9, 1 + 3) = \text{Gamma}(12, 4)$, mean 3 failures/month.
- The grid reproduces it. Use it to **validate** approximations on problems with known answers.

The limit of grids (dimensionality)

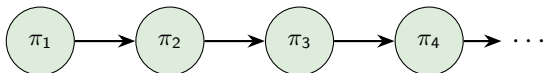


- 100 grid values per parameter.
- 1 parameter: 100 points. OK.
- 3 parameters: 10^6 points.
- 6 parameters: 10^{12} points. Impossible.

Real reliability and risk models often have **dozens** of parameters.

We need a method that explores only where the posterior is large.

MCMC

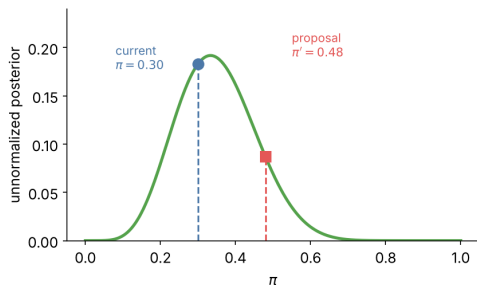


- **Markov chain:** each new value depends *only* on the current one.
- The walk is designed so that it **visits π in proportion to posterior density**: high-probability regions get many visits, low-probability regions few.
- The chain is a list of *dependent* draws, but their histogram approximates the posterior.

Analogy

An inspector walks a terrain where height = plausibility of π . She mostly stays on high ground, and occasionally steps into valleys. Time spent at each location $\approx$ posterior probability.

One Metropolis step



Only the **ratio** of heights matters,
so the hard denominator cancels!

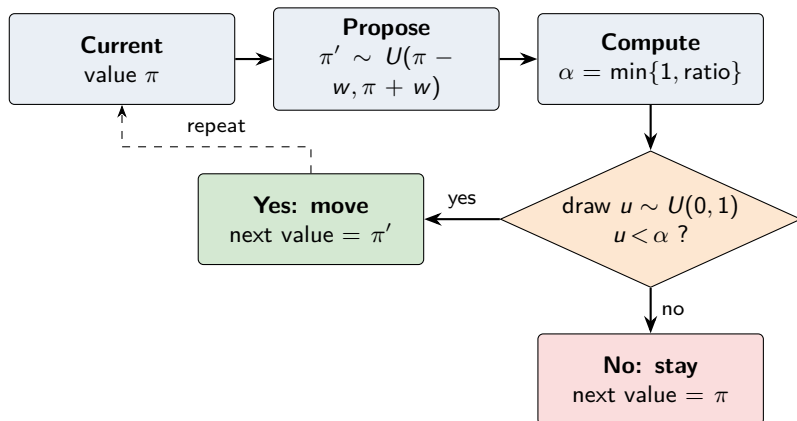
$$\alpha = \min \left\{ 1, \frac{f(\pi')L(\pi')}{f(\pi)L(\pi)} \right\}$$

Here:

$$\alpha = \frac{0.087}{0.183} \approx 0.47$$

- Uphill proposal: always accepted ($\alpha = 1$).
- Downhill: accepted with probability α .

The Metropolis algorithm



w is the **step size**. Too small: slow exploration. Too large: many rejections. See "lecture7_manual_mcmc_example.pdf"

Metropolis in R (and 15 lines)

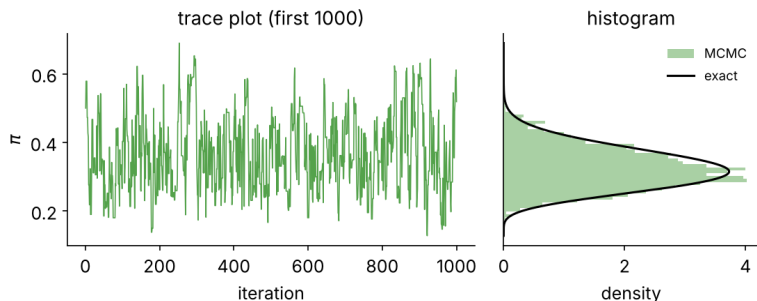
```
target <- function(pi) {                # prior x likelihood
  if (pi < 0 || pi > 1) return(0)      # outside (0,1): impossible
  dbeta(pi, 2, 8) * dbinom(5, 10, pi)
}

metropolis <- function(N, w, start) {
  chain <- numeric(N); current <- start
  for (i in 1:N) {
    proposal <- runif(1, current - w, current + w)      # (a)
    alpha <- min(1, target(proposal) / target(current)) # (b)
    if (runif(1) < alpha) current <- proposal            # (c)
    chain[i] <- current                                  # (d)
  }
  chain
}

chain <- metropolis(N = 5000, w = 0.2, start = 0.5)
```

(a) propose a nearby value **(b)** compare heights, the normalizing constant cancels **(c)** accept with probability α , else stay **(d)** record the current value, even if we stayed.

The chain



- **Trace plot:** the chain wanders around 0.35 with no long-term drift.
- **Histogram:** matches the exact Beta(7, 13) density (black).
- Acceptance rate here: about 66%.

In practice: rstan

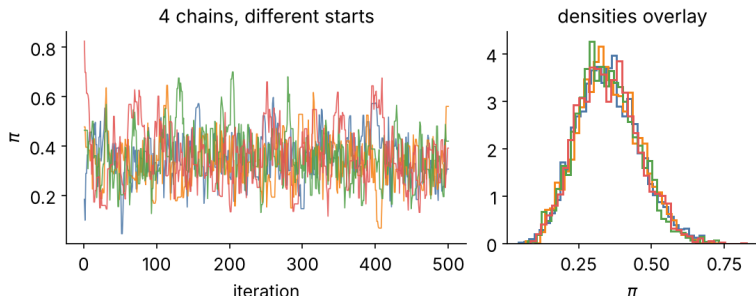
You rarely write your own sampler. **DEFINE** the model, then **SIMULATE**.

```
library(rstan)
# DEFINE: Stan syntax = data, parameters, model
bb_model <- "
  data { int<lower = 0, upper = 10> Y; }
  parameters { real<lower = 0, upper = 1> pi; }
  model {
    Y ~ binomial(10, pi);    // likelihood
    pi ~ beta(2, 8);        // prior
  }
"
# SIMULATE: 4 chains, 5000 iterations each (half is warm-up)
bb_sim <- stan(model_code = bb_model, data = list(Y = 5),
               chains = 4, iter = 5000 * 2, seed = 84735)
```

- The model block is the **same math** as before: likelihood and prior.
- Warm-up draws are thrown away; `bb_sim` keeps $4 \times 2500 = 10,000$ draws.

Four chains: first visual check

```
library(bayesplot)
mcmc_trace(bb_sim, pars = "pi", size = 0.1)      # trace plot
mcmc_dens_overlay(bb_sim, pars = "pi")           # density overlay
```



Chains start at *different* places, quickly mix into the same band (the “hairy caterpillar”), and give the same density. That is what healthy looks like.

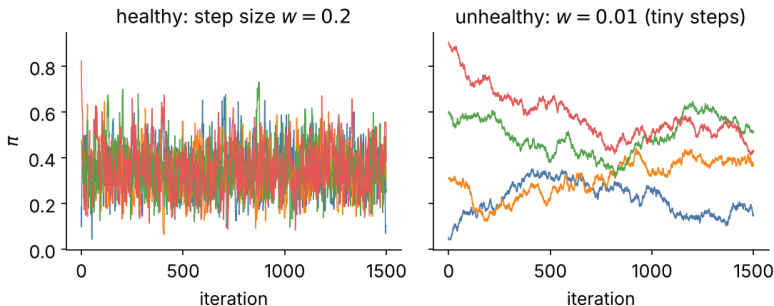
(Figure drawn with the Metropolis sampler; Stan output looks the same.)

How do we know the chain is trustworthy?

Check	Question	Healthy sign
Trace plot	Do chains mix and stay stable?	Overlapping “hairy caterpillars”
Density overlay	Do chains agree on the shape?	Curves on top of each other
Effective sample size ratio	How many <i>independent</i> draws are our draws worth?	> 0.1
Autocorrelation	Does the chain forget the past quickly?	Drops to ≈ 0 in a few lags
$\hat{R}$ (R-hat)	Do chains agree: between-chain vs within-chain spread	≈ 1 (< 1.05)

A bad simulation is like an uncalibrated sensor: the numbers look plausible but are wrong. Always check before you use them.

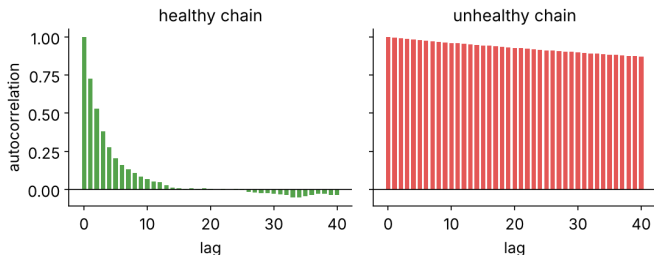
Healthy vs unhealthy chains



- **Left:** 4 chains with $w = 0.2$ forget their start and overlap.
- **Right:** with $w = 0.01$ each chain crawls and is still near its own start: the chains disagree. Results would depend on where we began.

Autocorrelation and effective sample size

```
mcmc_acf(bb_sim, pars = "pi")    # autocorrelation by lag  
neff_ratio(bb_sim, pars = "pi")  # effective size / actual size
```



Left: correlation fades after a few lags. **Right:** neighbours are nearly copies of each other. Out of 4500 draws, the healthy chain is worth ≈ 800 independent draws, the unhealthy one ≈ 9 .

R-hat: do the chains agree?

$$\hat{R} \approx \sqrt{\frac{\text{total variability (within + between chains)}}{\text{typical within-chain variability}}}$$

```
rhat(bb_sim, pars = "pi")      # want < 1.05
```

Healthy run ($w = 0.2$)

$\hat{R} \approx 1.00$
chains agree

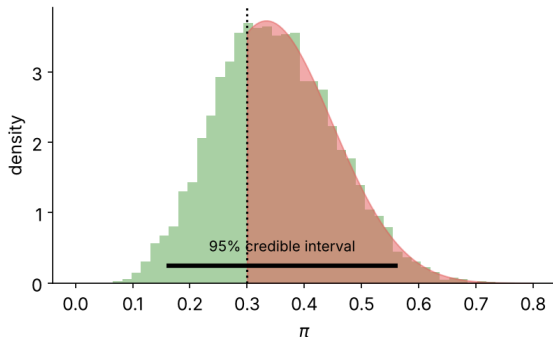
Unhealthy run ($w = 0.01$)

$\hat{R} \approx 1.36$
chains disagree

- If chains explore the same distribution, between-chain spread $\approx$ within-chain spread, so $\hat{R} \approx 1$.
- $\hat{R}$ far above 1: run longer, change the step size or fix the model.

Use the draws (risk numbers)

```
pi_draws <- as.data.frame(bb_sim, pars = "pi")$pi
mean(pi_draws)                # about 0.35
mean(pi_draws > 0.3)          # about 0.66
quantile(pi_draws, c(0.025, 0.975)) # about (0.16, 0.57)
```



Every question becomes counting draws. The conclusion matches the exact answer (0.67; (0.16, 0.57)): **replace** if the cost of a failure is high relative to replacement.

Grid vs MCMC

	Grid approximation	MCMC
Idea	Evaluate the posterior on a fixed grid	Random walk that visits π in proportion to the posterior
Draws	Independent	Dependent (check ESS)
Few parameters	Excellent, simple	Works
Many parameters	Fails (grid explodes)	Scales
Needs diagnostics?	No, just grid fineness	Yes: trace, ESS, $\hat{R}$
Tools	Base R	rstan, bayesplot

*Approximate when you must, **validate** whenever you can, and always finish with a decision-relevant number such as $P(\text{risk} > \text{threshold})$.*

Some practice problems

1. **Grid:** Repeat the example with prior $\text{Beta}(1, 1)$ and $Y = 2$ failures in $n = 20$. Use 6, 50 and 500 grid points. How does $P(\pi > 0.2 \mid y)$ change?
2. **Loss:** Change the cost of a failure to 60 and replacement to 30. At what posterior mean does the decision flip?
3. **Step size:** Run `metropolis()` with $w = 0.001, 0.05, 0.5, 5$. Plot the traces and compute the acceptance rate. Which is best?
4. **Diagnostics:** Run 4 chains from starts 0.05, 0.3, 0.6, 0.9 with $w = 0.005$. What do trace plots and $\hat{R}$ say? How many iterations fix it?
5. **Poisson:** Replace the Beta-Binomial Stan model by a Gamma-Poisson one for the monthly failure data and compare with $\text{Gamma}(12, 4)$.